

Node.js ile Web Programlama

DÖNEM PROJESİ TEKNİK RAPORU

Senaryo 2: Düşme ve Hareketsizlik Tespiti

Mobil Sensör Tabanlı Gerçek Zamanlı Güvenlik Platformu — CatchMe

Alan	Bilgi
Ders	Node.js ile Web Programlama
Senaryo	Senaryo 2 – Düşme ve Hareketsizlik Tespiti
Tarih	12 Haziran 2026
Danışman	Doç. Dr. İzzet Fatih ŞENTÜRK / Arş. Gör. Yusuf KAYIPMAZ
Hazırlayanlar	Hasna Şahinoğlu, Sudenur Elmas, Halime Buse Yalçın
Github Linki	https://github.com/Buse25/fall-detection

İçindekiler.....	3
Giriş.....	5
1. Gereksinim Analizi.....	5
1.1 Fonksiyonel Gereksinimler.....	5
1.2 Fonksiyonel Olmayan Gereksinimler.....	6
1.3 Proje Kapsamı.....	6
2. Kullanım Senaryosu.....	6
2.1 Birincil Kullanım Senaryosu: Düşme Tespiti ve Alarm.....	7
2.2 İkincil Kullanım Senaryosu: Hareketsizlik Tespiti.....	7
2.3 Acil Durum Kişisi ve Profil Yönetimi.....	7
2.4 Admin Senaryosu: Web Panel ile Gerçek Zamanlı İzleme.....	7
3. Sistem Mimarisi.....	8
3.1 Katman Yapısı.....	8
3.2 Genel Sistem Akışı.....	8
3.3 Mobil Sensör Parametreleri.....	8
3.4 Kurulum ve Çalıştırma.....	9
1. AI Mikroservisi (Python/FastAPI).....	9
2. Backend (Node.js/Express).....	9
3. Web Yönetim Paneli (React/Vite).....	10
4. Mobil İstemci (React Native/Expo).....	10
4. Kullanılan Teknolojiler.....	10
4.1 Backend.....	10
4.2 AI Mikroservisi.....	11
4.3 Mobil Uygulama.....	11
4.4 Web Yönetim Paneli.....	12
4.5 Geliştirme ve Test.....	12
5. Veri Modeli.....	12
5.1 Varlık-İlişki Özeti.....	12
5.2 User Modeli.....	13
5.3 SensorData Modeli.....	13
5.4 Alarm Modeli.....	13
5.5 Device Modeli.....	14
5.6 Redis Durum Anahtarları.....	14
6. Gerçekleştirilen Modüller.....	14
6.1 Kullanıcı Doğrulama ve Yetkilendirme Modülü.....	15
6.2 Sensör Veri Toplama ve İletim Modülü.....	15
6.3 Düşme Tespit (AI Destekli Anomali Tespiti) Modülü.....	15
6.3.1 Birincil Yöntem: AI Modeli.....	15
6.3.2 İkincil Yöntem: Kural Tabanlı Fallback.....	16
6.3.3 Düşme Durum Makinesi (Fall State Machine).....	16
6.4 Hareketsizlik Tespiti Modülü.....	16
6.5 Alarm ve Bildirim Modülü.....	17
6.6 Socket.IO Gerçek Zamanlı İletişim Modülü.....	17
6.7 Web Yönetim Paneli (Admin Dashboard) Modülü.....	18
6.7.1 Giriş Sayfası (LoginPage).....	18
6.7.2 Dashboard Sayfası (DashboardPage).....	18

6.7.3 Sensör Grafikleri (SensorChart).....	18
6.7.4 Cihaz Listesi Sayfası (DevicesPage).....	18
6.7.5 Alarm Geçmişi Sayfası (AlarmHistoryPage).....	18
6.7.6 Profil Sayfası (ProfilePage).....	19
6.7.7 Panel REST API Uç Noktaları.....	19
6.8 AI Mikroservisi Modülü.....	19
7. API Dokümantasyonu.....	19
7.1 Kimlik Doğrulama (/api/auth).....	20
7.2 Sensör Verisi (/api/sensor-data).....	20
7.3 Alarm Yönetimi (/api/alarms).....	20
7.4 Admin Uç Noktaları (/api/admin).....	20
7.5 Web Panel Uç Noktaları (/api/panel).....	20
7.6 AI Servisi Uç Noktaları (Port 8000).....	21
7.7 Sağlık Kontrolü.....	21
8. Test Süreci.....	21
8.1 Backend Test Sonuçları.....	21
8.2 Diğer Bileşenler İçin Test Kapsamı.....	22
9. Karşılaşılan Kısıtlar ve Geliştirme Önerileri.....	22
9.1 Mevcut Kısıtlar.....	22
9.2 Gelecek Geliştirme Önerileri.....	22
10. Sonuç.....	23
Tamamlanan İleri Düzey Özellikler.....	23

Giriş

Günümüzde düşme olayları, özellikle yaşlı bireyler ve yalnız yaşayan kişiler için ciddi sağlık riskleri oluşturmaktadır. Acil müdahale süresinin kısaltılması, olayın otomatik tespitine ve yetkili kişilere anında bildirim yapılmasına bağlıdır. Bu kapsamda geliştirilen platform, akıllı telefonun ivmeölçer ve jiroskop sensörlerini birer IoT uç düğümü olarak kullanmakta; toplanan veriler Node.js tabanlı bir backend'e iletilerek gerçek zamanlı analiz ve anomali tespiti yapılmaktadır. Platform; makine öğrenmesi tabanlı düşme tespiti, hareketsizlik izleme, durum makinesi (state machine) tabanlı doğrulama ve gerçek zamanlı bir web yönetim paneli ile uçtan uca çalışan bir güvenlik ekosistemi sunmaktadır.

Fall-Detection projesinin geliştirme sürecinde görev dağılımı, şu şekilde yürütülmüştür: Halime Yalçın, projenin temel backend altyapısını kurarak MongoDB entegrasyonu, kimlik doğrulama (auth) sistemi, kural tabanlı düşme tespiti ve alarm uç noktalarının (endpoints) geliştirilmesini üstlenmiştir. Sudenur Elmas, kullanıcı arayüzü ve mobil uygulama süreçlerine odaklanarak mobil uygulamanın temel yapısı ve tasarımını, socket/token yönetimini, web panel arayüzünü, API entegrasyonlarını ve acil durum kişileri gibi ön yüz özelliklerini hayata geçirmiştir. Hasna Şahinoğlu ise yapay zeka modelinin sisteme entegrasyonu, Redis ile iki aşamalı düşme ve hareketsizlik tespiti algoritmaları, WebSocket haberleşmesi ve web panel geliştirmelerinin yanı sıra, projenin mobil, web ve yapay zeka modüllerinin test süreçlerini ve monorepo dosya mimarisini yürütmüştür.

1. Gereksinim Analizi

1.1 Fonksiyonel Gereksinimler

- Kullanıcılar sisteme kayıt olabilmeli, giriş yapabilmeli ve JWT tabanlı kimlik doğrulaması ile korunan kaynaklara erişebilmelidir.
- Mobil uygulama, ivmeölçer ve jiroskop sensörlerinden 50 Hz frekansında veri toplamalı; bu verileri zaman damgasıyla birlikte sunucuya iletmelidir.
- Backend, gelen sensör verilerini doğrulamalı, depolamalı ve düşme tespiti için analiz edebilmelidir.
- Sistem, düşme tespiti durumunda AI modeli ve durum makinesi doğrulaması ile otomatik olarak yüksek öncelikli alarm kaydı oluşturmalıdır.
- Kullanıcılar mobil tarafta aktif bir alarmı “İyiyim” diyerek iptal edip çözüldü olarak işaretleyebilmelidir.
- Her kullanıcı acil durum kişileri (ad, telefon) tanımlayabilmeli ve profil bilgilerini (profil tipi, uyku takvimi) yönetebilmelidir.
- Sistem, kullanıcı hareketsizliğini izlemeli ve gece/gündüz dinamik eşiklerine göre hareketsizlik alarmı üretebilmelidir.
- Admin rolüne sahip kullanıcılar, web paneli üzerinden tüm kullanıcıları, cihazları, düşme kayıtlarını, sensör grafiklerini ve özet istatistikleri gerçek zamanlı olarak görüntüleyebilmelidir.

1.2 Fonksiyonel Olmayan Gereksinimler

- Sistem, Socket.IO aracılığıyla gerçek zamanlı veri akışını desteklemelidir.
- Tüm şifreler bcryptjs ile hashlenerek saklanmalı; JWT token süresi kontrol altında tutulmalıdır.
- API uç noktaları express-validator ile girdi doğrulamasına tabi tutulmalı; hatalı isteklere anlamlı HTTP hata yanıtları döndürülmelidir.
- Uygulama helmet ve express-rate-limit middleware'leri ile temel güvenlik önlemlerine sahip olmalıdır.
- AI mikroservisi 3 saniyelik zaman aşımı ile çağrılmalı; erişilemezse kural tabanlı fallback otomatik devreye girmelidir.
- Redis, kısa ömürlü durum verilerini (düşme durum makinesi ve sensör buffer'ı) TTL ile yönetmelidir; hareketsizlik durum bilgileri ise kalıcı anahtarlar olarak tutulur ve durum makinesi tarafından açıkça güncellenir.
- Kritik işlevler otomatik testlerle doğrulanmalıdır. Backend, Web paneli (Vitest), mobil uygulama (Jest) ve AI servisi (pytest) için ayrı test süitleri oluşturulmuştur.

1.3 Proje Kapsamı

Proje, proje tanım belgesinde belirlenen zorunlu modüllerin tümünü içermektedir: mobil veri toplama, Node.js tabanlı arka uç, kullanıcı kimlik doğrulama ve yetkilendirme, veritabanı yönetimi, gerçek zamanlı veri akışı altyapısı ve admin paneli, anomali/düşme tespiti ve alarm mekanizması. Bunların yanı sıra aşağıdaki ileri düzey özellikler de gerçekleştirilmiştir:

- AI Mikroservisi: Python/FastAPI tabanlı, Random forest modeli kullanan bağımsız bir düşme tespit servisi.
- Hareketsizlik Tespiti: Redis tabanlı durum makinesi ile gece/gündüz dinamik eşikli hareketsizlik izleme.
- Düşme Durum Makinesi: AI tahmini ve varyans doğrulamasını birleştiren çok aşamalı düşme onay mekanizması (NORMAL → IMPACT_DETECTED → FALL_CONFIRMED).
- Çevrimdışı Veri Tamponlama: Mobil uygulamada bellek içi offline kuyruk mekanizması.
- Jiroskop Entegrasyonu: Jiroskop verisi hem AI modeline girdi olarak kullanılmakta hem de web panelinde görselleştirilmektedir.

2. Kullanım Senaryosu

Geliştirilen platform, öncelikli olarak yaşlı bireyler, yalnız yaşayan kişiler ve bakım desteğine ihtiyaç duyan kullanıcılar için tasarlanmıştır. Aşağıda sistemin uçtan uca kullanım akışı açıklanmaktadır.

2.1 Birincil Kullanım Senaryosu: Düşme Tespiti ve Alarm

Kullanıcı mobil uygulamayı açıp giriş yaptığında sensör izleme başlatılır. Uygulama arka planda ivmeölçer ve jiroskop verilerini toplar; her 75 örnekten oluşan yaklaşık 1.5 saniyelik pencere Socket.IO üzerinden backend'e iletilir. Backend, gelen pencereyi AI mikroservisine yönlendirir. AI modeli, pencereden çıkardığı 6 istatistiksel öznitelik üzerinden düşme olasılığını hesaplar. Olasılık $\geq 85\%$ eşikini aştığında durum makinesi IMPACT_DETECTED aşamasına geçer; ardından düşük varyans doğrulamasıyla düşme onaylanır (FALL_CONFIRMED) ve yüksek öncelikli alarm kaydı oluşturulur. Kullanıcıya mobil uygulamada geri sayımlı bir bildirim gösterilir; "İyiymi" butonuyla alarmı iptal edebilir.

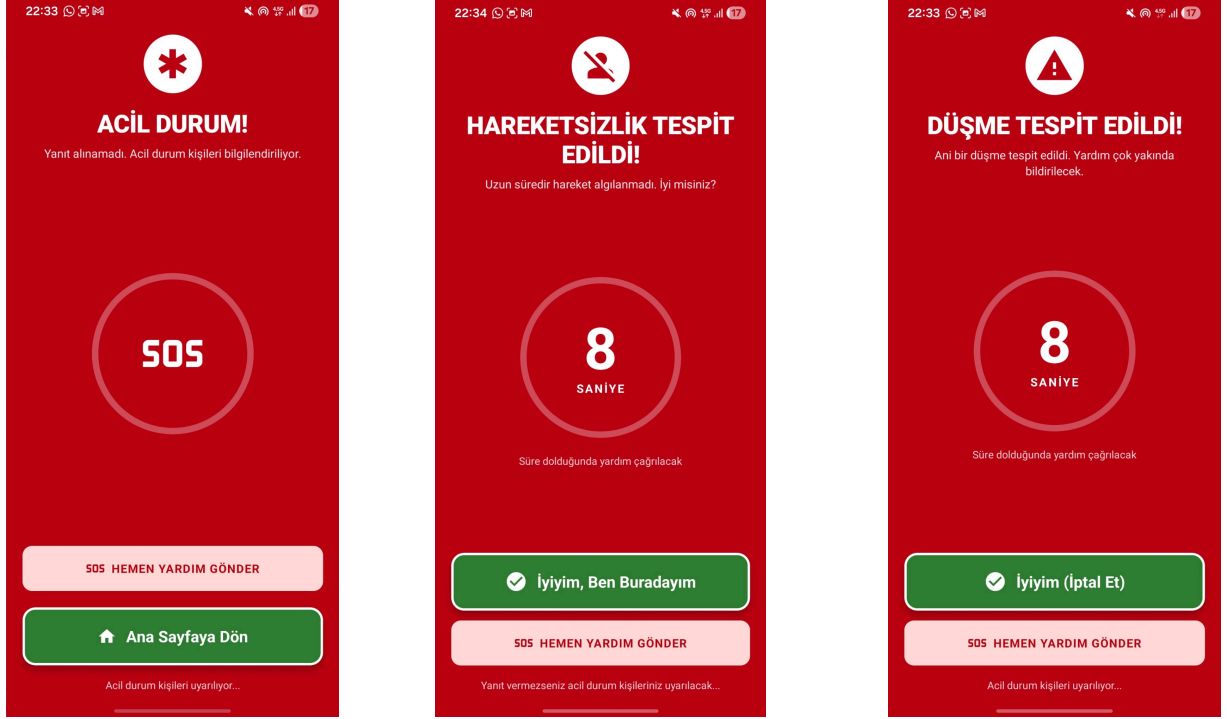


Bu gerçek zamanlı Socket.IO akışına ek olarak, backend'de basit bir REST uç noktası olan POST /api/sensor-data da bulunmaktadır. Bu uç nokta, AI ve durum makinesi mantığından bağımsız olarak yalnızca magnitude > 2.5 g kuralına göre çalışır ve eşik aşıldığında doğrudan alarm üretir. Bu iki yol, sistemin farklı senaryolarda (gerçek zamanlı izleme ve basit toplu kayıt) çalışabilmesini sağlayan tamamlayıcı yapılardır; AI servisine 3 saniye içinde ulaşılmadığında ise socket akışı da aynı kural tabanlı eşik (magnitude > 2.5 g) otomatik olarak düşer.

Şekil 1: Mobil Arayüz Ana Sayfası

2.2 İkincil Kullanım Senaryosu: Hareketsizlik Tespiti

Sistem, kullanıcının ivmeölçer varyansını sürekli izler. Kullanıcının uyku takvimine göre dinamik eşikler belirlenir (gündüz: 2 saat, gece: 8 saat). Hareketsizlik süresi eşiği aştığında NORMAL → PRE_ALARM geçişi yapılarak kullanıcıya ön alarm gönderilir. Kullanıcı varsayılan olarak 60 saniye içinde “İyiyim, Ben Buradayım” butonuna basmazsa PRE_ALARM → CONFIRMED geçişi gerçekleşir ve acil durum alarmı oluşturulur. Geliştirme/demo ortamında bu sürenin daha kısa tutulabilmesi için .env dosyası üzerinden ayarlanabilir bir parametre olarak tasarlanmıştır.



Şekil 2: Alarm Ekranı

2.3 Acil Durum Kişisi ve Profil Yönetimi

← CatchMe

Hesabınızı Oluşturun

7/24 akıllı düşme tespiti ve tıbbi gözetim için CatchMe'ye katılın.

Profil Tipinizi Seçin
Bu seçim, hareketlilik kalıplarınıza göre düşme tespit algoritmalarını optimize eder.

Yaşlı Kullanıcı
Kırılgan denge için yüksek hassasiyetli tespit.

Genç/Aktif
Yüksek etkili aktiviteler için dengeli tespit.

Ad Soyad
Adınızı ve soyadınızı girin

E-posta
E-posta adresinizi girin

Şifre
En az 8 karakter

☐ Gizlilik Politikasını kabul ediyorum ve veri toplamamın farkında olduğumu beyan ediyorum.

Kayıt Ol

Kullanıcı, mobil uygulamadaki ayrı bir “Acil Durum Kişisi” sekmesi üzerinden acil durum kişinin adını ve telefon numarasını tanımlar. Profil sayfasında ise kullanıcı adı, profil tipi ve uyku takvimi (gece başlangıç-bitiş saatleri) yönetilir. Backend'de profil tipi yaşlı/genç olarak tanımlanmıştır; AI modeline yaş grubu bilgisi bu iki kategoriye göre iletilir. Uyku takvimi ayarı, hareketsizlik eşiklerinin kişiselleştirilmesinde kullanılır.

Şekil 3: Mobil Arayüz Profil Ekranı

2.4 Admin Senaryosu: Web Panel ile Gerçek Zamanlı İzleme

Admin rolüne sahip kullanıcılar, React tabanlı web paneline giriş yaparak merkezi bir dashboard üzerinden sistemi izleyebilmektedir. Dashboard'da toplam alarm sayısı, bugünkü düşme sayısı ve toplam sensör kaydı sayısı istatistik kartları olarak gösterilir; çözülmemiş alarm sayısı ise üst menüdeki bildirim alanında ayrıca belirtilir. Sensör grafiklerinde ivmeölçer ve jiroskop büyüklüğü Recharts ile zaman serisi olarak görselleştirilir. Cihaz listesi sayfasında tüm cihazlar; durumları, son magnitude değerleri ve düşme sayılarıyla birlikte izlenir. Alarm geçmiş sayfasında alarmlar tipine, ciddiyetine ve durumuna göre filtrelenebilir ve listelenir. Socket.IO üzerinden düşme ve hareketsizlik olayları panele anlık olarak iletilir.

3. Sistem Mimarisi

Platform, beş ana bileşenden oluşan, mikroservis destekli katmanlı (layered) bir mimari üzerine inşa edilmiştir. Bu yaklaşım her bileşenin bağımsız olarak geliştirilebilmesini, test edilebilmesini ve ölçeklendirilebilmesini sağlamaktadır.

3.1 Katman Yapısı

Katman	Teknoloji	Sorumluluk
Sunum Katmanı (Mobil)	React Native / Expo (TypeScript)	Sensör toplama, kullanıcı etkileşimi
Sunum Katmanı (Web)	React / Vite / Tailwind CSS	Admin dashboard, grafik görselleştirme
API ve İş Mantığı Katmanı	Node.js / Express.js	REST API, Socket.IO, durum makineleri
AI Katmanı	Python / FastAPI / scikit-learn	Makine öğrenmesi ile düşme tespiti
Veri Katmanı	MongoDB (Mongoose) + Redis	Kalıcı veri depolama + anlık durum yönetimi

Sistemde iki ayrı veri yolu bulunmaktadır: gerçek zamanlı ve asıl tespit pipeline'ı Socket.IO üzerinden iletilen sensor_window paketleriyle çalışır (AI servisi + Redis durum makineleri); REST tarafındaki POST /api/sensor-data ise yalnızca kural tabanlı eşik kontrolü yapan, daha basit ve bağımsız bir kayıt/alarm yoludur.

3.2 Genel Sistem Akışı

Mobil uygulama, topladığı sensör pencerelerini Socket.IO üzerinden backend'e iletir. Backend, gelen pencereyi AI servisine yönlendirir; AI servisten gelen sonuç Redis tabanlı düşme durum makinesi ile değerlendirilir. Aynı backend, hareketsizlik durumunu da ayrı bir Redis tabanlı durum makinesiyle takip eder. Alarm oluştuğunda hem mobil uygulamaya hem de web paneline Socket.IO event'leri ile bildirim gönderilir. Web paneli, /api/panel altındaki uç noktalar üzerinden istatistik, alarm, cihaz ve sensör grafiği verilerini çeker.

3.3 Mobil Sensör Parametreleri

Parametre	Değer
Örnekleme Aralığı	20 ms (50 Hz)
Pencere Boyutu	75 örnek (~1.5 sn)
Sensörler	İvmeölçer + Jiroskop
İletim Protokolü	Socket.IO (WebSocket)
Çevrimdışı Kuyruk	Bellek içi, maks. 100 pencere (FIFO)
Reconnect sonrası	Otomatik kuyruk boşaltma

Bağlantı kesilmesi durumunda sensör pencereleri bellek içi bir kuyrukta tutulur (maksimum 100 pencere); bağlantı yeniden kurulduğunda kuyruk otomatik olarak boşaltılır. Mobil arayüzde bağlantı durumu çevrimiçi/çevrimdışı rozetiyle anlık olarak gösterilir.

3.4 Kurulum ve Çalıştırma

Projeyi kendi bilgisayarınızda local olarak çalıştırmak için, sisteminizde Node.js, Python, MongoDB ve Redis servislerinin kurulu ve aktif olduğundan emin olun. Ardından adım adım aşağıdaki talimatları izleyin:

1. AI Mikroservisi (Python/FastAPI)

AI mikroservisinin aktif olması, modelin doğru sonuç verebilmesi için gereklidir (Varsayılan port: 8000).

```
cd ai-service
```

Gerekli bağımlılıkların yüklenmesi:

```
pip install -r requirements.txt
```

Mikroservisi başlatma:

```
uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

2. Backend (Node.js/Express)

Ana iş mantığını ve soket sunucusunu barındıran yapıdır (Varsayılan port: 5000).

```
cd backend
```

Gerekli paketlerin yüklenmesi:

```
npm install
```

.env dosyasını oluşturun (ya da yukarıdaki şablonu içine kopyalayın)


```
cp .env.example .env
```

Sunucuyu geliştirici modunda başlatma:

```
npm run dev
```

veya standart başlangıç: `npm start`

3. Web Yönetim Paneli (React/Vite)

Adminlerin gerçek zamanlı izleme yapabildiği panel katmanıdır.

```
cd web-panel
```

Gerekli paketlerin yüklenmesi:

```
npm install
```

Geliştirme sunucusunu başlatma:

```
npm run dev
```

4. Mobil İstemci (React Native/Expo)

Uygulamayı fiziksel bir telefonda çalıştırmak için "Expo Go" uygulamasını kullanabilirsiniz.

```
cd mobile
```

Gerekli bağımlılıkların yüklenmesi:

```
npm install
```

Uygulamayı başlatma:

```
npx expo start
```

(Komut sonrası terminalde çıkan QR kodu, cihazınızdaki Expo Go kamerasına okutarak test sürecine başlayabilirsiniz.)

4. Kullanılan Teknolojiler

4.1 Backend

Teknoloji	Kullanım Alanı
Node.js	Sunucu çalışma zamanı
Express.js	HTTP sunucusu ve yönlendirme
Mongoose	MongoDB nesne modelleme (ODM)

Socket.IO	Gerçek zamanlı çift yönlü iletişim
jsonwebtoken	JWT oluşturma ve doğrulama
bcryptjs	Şifre hashleme
express-validator	Girdi doğrulama
helmet	HTTP güvenlik başlıkları
express-rate-limit	İstek hız sınırlama
axios	AI servisine HTTP istekleri
redis	Durum makinesi ve buffer yönetimi
dotenv	Ortam değişkeni yönetimi
cors	Cross-Origin Resource Sharing

4.2 AI Mikroservisi

Teknoloji	Kullanım Alanı
Python 3.12+	Programlama dili
FastAPI	ASGI web çerçevesi
uvicorn	ASGI sunucusu
scikit-learn	Makine öğrenmesi modeli (eğitim/tahmin)
numpy / pandas	Sayısal hesaplama ve veri çerçevesi
pydantic	İstek/yanıt veri doğrulama

4.3 Mobil Uygulama

Teknoloji	Kullanım Alanı
React Native	Mobil uygulama çerçevesi
Expo	Geliştirme platformu
Expo Router	Dosya tabanlı navigasyon
expo-sensors	İvmeölçer ve jiroskop erişimi
AsyncStorage	Yerel veri depolama (JWT kalıcılığı)
Socket.IO Client	Gerçek zamanlı sunucu iletişimi
TypeScript	Tip güvenli geliştirme
react-native-reanimated	Animasyonlar
react-native-gesture-handler	Dokunma hareketleri

4.4 Web Yönetim Paneli

Teknoloji	Kullanım Alanı
React	UI bileşen kütüphanesi
Vite	Geliştirme ve derleme aracı
Tailwind CSS	Utility-first CSS çerçevesi
React Router DOM	Sayfa yönlendirme
Recharts	Grafik ve veri görselleştirme
Axios	HTTP istemcisi
Socket.IO Client	Gerçek zamanlı panel güncellemeleri

4.5 Geliştirme ve Test

Teknoloji	Kullanım Alanı
Jest + Supertest	Backend birim/entegrasyon testleri
mongodb-memory-server	Backend testleri için bellek içi MongoDB
Vitest	Web paneli testleri
Jest (jest-expo)	Mobil uygulama testleri
pytest + httpx	AI servisi testleri
nodemon	Otomatik yeniden başlatma (geliştirme)
ESLint	Kod kalite kontrolü

Test altyapısı tek bir bileşenle sınırlı değildir: backend Jest/Supertest ile, web paneli Vitest ile, mobil uygulama jest-expo ile, AI servisi ise pytest ve httpx ile test edilmektedir. Detaylar Bölüm 8'de verilmiştir.

5. Veri Modeli

Veri modeli, sistemin temel varlıklarını ve aralarındaki ilişkileri tanımlamaktadır. MongoDB'nin belge tabanlı yapısı esnek şema tasarımına olanak tanırken, Mongoose katmanı veri bütünlüğünü ve doğrulamayı güvence altına almaktadır. Redis ise anlık durum verilerini (düşme durumu, hareketsizlik durumu, sensör buffer'ı) yönetmektedir.

5.1 Varlık-İlişki Özeti

User —1:N—► SensorData

User —1:N—► Alarm

SensorData —1:1—► Alarm (sensorDataId referansı)

Device —N:1—► User (userId referansı, cihaz-kullanıcı eşleştirmesi)

5.2 User Modeli

Alan	Tip	Zorunlu	Varsayılan	Açıklama
name	String	✓	-	Kullanıcı adı
email	String	✓ (unique)	-	E-posta adresi (küçük harf)
password	String	✓ (min 6)	-	Hashlenmiş şifre
role	String (enum)	-	"user"	"user" "admin"
profileType	String (enum)	-	"other"	"elderly" "worker" "athlete" "other"
emergencyContactName	String	-	""	Acil durum kişisi adı
emergencyContactPhone	String	-	""	Acil durum kişisi telefonu
sleepSchedule.nightStart	String	-	"23:00"	Gece başlangıç saati (HH:MM)
sleepSchedule.nightEnd	String	-	"07:00"	Gece bitiş saati (HH:MM)
createdAt / updatedAt	Date	oto	-	Mongoose timestamps

Not: Mobil uygulamada acil durum kişisi yönetimi tek bir kayıtla sınırlıdır (ad + telefon). Birden fazla acil durum kişisi tanımlama, ileri sürüm için planlanmış bir geliştirmedir.

5.3 SensorData Modeli

Alan	Tip	Zorunlu	Varsayılan	Açıklama
userId	String	✓	-	Veri sahibi kullanıcı
deviceId	String	✓	-	Kaynak cihaz tanımlayıcı
timestamp	Date	-	Date.now	Ölçüm zaman damgası
accelerometer.x/y/z	Number	✓	-	İvme bileşenleri (g)
accelerometer.magnitude	Number	-	-	Bileşke ivme büyüklüğü ($\sqrt{x^2+y^2+z^2}$)
gyroscope.x/y/z	Number	✓	-	Açısal hız bileşenleri (rad/s)
isFallDetected	Boolean	-	false	Düşme tespit edildi mi
fallScore	Number	-	0	AI olasılık skoru veya magnitude
detectionMethod	String (enum)	-	"rule-based"	"rule-based" "ai-model"
createdAt / updatedAt	Date	oto	-	Mongoose timestamps

5.4 Alarm Modeli

Alan	Tip	Zorunlu	Varsayılan	Açıklama
userId	String	✓	-	Alarm sahibi kullanıcı
deviceId	String	✓	-	Alarmı tetikleyen cihaz
sensorDataId	ObjectId (ref)	✓	-	İlişkili SensorData referansı
alarmType	String (enum)	-	"fall"	"fall" "inactivity" "anomaly"
severity	String (enum)	-	"high"	"low" "medium" "high"
message	String	-	"Fall detected"	Alarm detay mesajı
isResolved	Boolean	-	false	Çözüldü mü
resolvedAt	Date	-	null	Çözüm zaman damgası
createdAt / updatedAt	Date	oto	-	Mongoose timestamps

5.5 Device Modeli

Sensör verisi gönderen her cihaz, socket bağlantısı üzerinden upsert edilerek Device koleksiyonunda tutulur. Bu model; cihazın bağlı olduğu kullanıcı (userId, ObjectId referansı), son görülme zamanı, son magnitude değeri ve toplam düşme sayısı gibi alanları içerir ve /api/panel/devices uç noktası tarafından kullanılır. (Not: SensorData modelindeki userId alanı String tipinde tutulurken, Device modelinde userId ObjectId referansı olarak tanımlanmıştır; bu, iki modelin farklı amaçlarla — ham veri kaydı ve ilişkisel cihaz takibi — tasarlanmış olmasından kaynaklanmaktadır.)

5.6 Redis Durum Anahtarları

Anahtar Deseni	Değer	TTL	Açıklama
fall:state:{deviceId}	JSON {state, timestamp}	3 sn	Düşme durum makinesi (IMPACT_DETECTED)
fall:buffer:{deviceId}	List (float)	3 sn	Sensör varyans buffer'ı
inactivity:last_active:{deviceId}	ISO string	-	Son hareket zaman damgası
inactivity:state:{deviceId}	JSON {state, pre_alarm_start}	-	Hareketsizlik durum makinesi

Düşme tespitiyle ilgili anahtarlar (fall:state, fall:buffer) kısa ömürlü olup 3 saniyelik TTL ile otomatik temizlenir; bu, doğrulama penceresinin süresini sınırlamak için bilinçli bir tasarım kararıdır. Hareketsizlik anahtarları ise saatler sürebilen bir süreci takip ettiğinden TTL kullanılmaz; durumları, ilgili durum makinesi tarafından gerektiğinde açıkça güncellenir veya silinir.

6. Gerçekleştirilen Modüller

6.1 Kullanıcı Doğrulama ve Yetkilendirme Modülü

Sistem, JWT tabanlı stateless kimlik doğrulama mekanizmasıyla güvence altına alınmıştır. Kayıt sırasında şifreler bcryptjs ile hashlenir; giriş sonrası oluşturulan JWT token, sonraki isteklerde Authorization: Bearer başlığıyla iletilir ve mobil uygulamada AsyncStorage ile kalıcı olarak saklanır. Sistemde iki rol bulunur: user kendi sensör verilerini, alarm kayıtlarını ve profil bilgilerini yönetebilir; admin ise web paneli üzerinden tüm kullanıcıları, cihazları, alarmları ve sistem istatistiklerini görüntüleyebilir. Admin'e özel uç noktalar adminOnly middleware'i ile korunur; web panelinde ayrıca frontend tarafında da rol kontrolü yapılır ve admin olmayan bir token ile giriş denendiğinde kullanıcı otomatik olarak çıkış yaptırılıp login sayfasına yönlendirilir.

6.2 Sensör Veri Toplama ve İletim Modülü

Mobil uygulama, Expo Sensors API'si aracılığıyla ivmeölçer ve jiroskop sensörlerine 20 ms örnekleme aralığıyla erişir. Her 75 örnekten oluşan yaklaşık 1.5 saniyelik pencere, sensor_window eventi olarak Socket.IO bağlantısı üzerinden backend'e iletilir:

```
{  "userId": "string",  "deviceId": "string",  "windowStart": "ISO 8601",  "windowEnd": "ISO 8601",  "sampleRateHz": 50,  "readings": [    {      "timestamp": "ISO 8601",      "accelerometer": { "x": 0.0, "y": 0.0, "z": 0.0 },      "gyroscope": { "x": 0.0, "y": 0.0, "z": 0.0 }    }  ] }
```

Bağlantı kesilmesi durumunda veriler bellek içi bir kuyrukta tutulur (maksimum 100 pencere, FIFO); bağlantı yeniden kurulduğunda kuyruk otomatik olarak boşaltılır. Bu mekanizma kısa süreli ağ kesintilerinde veri kaybını önler. Bağlantı durumu mobil arayüzde çevrimiçi/çevrimdışı rozetiyle anlık olarak gösterilir.

6.3 Düşme Tespit (AI Destekli Anomali Tespiti) Modülü

Düşme tespiti iki katmanlı bir stratejiyle gerçekleştirilir:

6.3.1 Birincil Yöntem: AI Modeli

AI mikroservisi, Python/FastAPI üzerine inşa edilmiş bağımsız bir servistir ve Node.js backend'den aiService.js modülü aracılığıyla HTTP POST isteğiyle çağrılır. Gelen sensör penceresi üzerinden 6 istatistiksel öznitelik hesaplanır:

Öznitelik	Hesaplama	Açıklama
Yaş_Grubu	Profil tipinden türetilir	1 = yaşlı ("elderly"), 0 = Genç

Acc_Min	min(SMV_acc)	İvmeölçer SMV minimum değeri
Acc_Max	max(SMV_acc)	İvmeölçer SMV maksimum değeri
Acc_Var	var(SMV_acc)	İvmeölçer SMV varyansı
Gyro_Max	max(SMV_gyro)	Jiroskop SMV maksimum değeri (°/s)
Gyro_Var	var(SMV_gyro)	Jiroskop SMV varyansı (°/s)

SMV (Sinyal Büyüklük Vektörü) = $\sqrt{x^2 + y^2 + z^2}$. Jiroskop verileri Expo sensörlerinden rad/s biriminde alınır ve model eğitiminde derece/s kullanıldığından 57.2958 çarpanıyla dönüştürülür.

Tahmin: prediction = model.predict(features) → 0 (düşme yok) | 1 (düşme); probability = model.predict_proba(features) → düşme sınıfı olasılığı (0.0 – 1.0). Karar kuralı:

```
is_fall = (prediction == 1) AND (probability ≥ 0.85)
```

FALL_PROBABILITY_THRESHOLD = 0.85 eşiği, hem modelin düşme tahmin etmesini hem de bunu yüksek güvenle yapmasını gerektirir. Bu çift koşul yanlış pozitif oranını düşürmektedir. Kullanılan model bir Random Forest sınıflandırıcısıdır; AI servisi fall_model.pkl dosyasını yükleyip tahmin için kullanır.

6.3.2 İkincil Yöntem: Kural Tabanlı Fallback

AI servisine 3 saniye içinde ulaşamadığında veya hata oluştuğunda, kural tabanlı fallback otomatik olarak devreye girer:

```
magnitude =  $\sqrt{accX^2 + accY^2 + accZ^2}$  FALL_THRESHOLD = 2.5 g magnitude >
FALL_THRESHOLD → isFallDetected = true, detectionMethod = "rule-based" Alarm oluşturun:
{ alarmType: "fall", severity: "high" }
```

Bu yaklaşım, AI servisinin erişilemez olduğu durumlarda bile sistemin işlevselliğini sürdürmesini garanti eder.

6.3.3 Düşme Durum Makinesi (Fall State Machine)

AI modelinin is_fall = true üretmesi tek başına alarm oluşturmaz. Yanlış pozitifleri azaltmak için Redis tabanlı bir durum makinesi uygulanmıştır:

- NORMAL → IMPACT_DETECTED: AI modeli düşme tespit ettiğinde geçiş yapılır; Redis'e 3 saniyelik TTL ile kaydedilir.
- IMPACT_DETECTED → FALL_CONFIRMED: Sonraki pencerelerde ivme varyansı 0.5 g²'nin altında kalıyorsa (kişi hareketsiz kalmış) düşme onaylanır ve alarm oluşturulur.
- IMPACT_DETECTED → NORMAL: Varyans 1.5 g²'nin üzerindeyse (kişi hareket ediyor) durum sıfırlanır ve yanlış alarm iptal edilir.
- 0.5 g² ile 1.5 g² arasındaki varyans değerleri için bir geçiş tanımlanmamıştır; bu ara bölgede durum IMPACT_DETECTED olarak kalır ve bir sonraki pencerede yeniden değerlendirilir, ya da TTL süresi dolarak NORMAL'e döner.
- TTL Süresi Dolması: 3 saniye içinde doğrulama gelmezse Redis kaydı otomatik siler ve durum NORMAL'e döner.

6.4 Hareketsizlik Tespiti Modülü

Hareketsizlik tespiti, inactivityManager.js servisi tarafından Redis tabanlı bir durum makinesiyle gerçekleştirilir. Her sensör penceresinde ivme varyansı hesaplanır ve hareket durumu değerlendirilir.

Zaman Dilimi	Hareketsizlik Eşiği	Açıklama
Gündüz	2 saat (7200 sn)	Normal aktivite döneminde uzun hareketsizlik anormal
Gece	8 saat (28800 sn)	Uyku süresi boyunca hareketsizlik normal kabul edilir

Gece/gündüz ayrımı kullanıcının uyku takvimindeki nightStart ve nightEnd saatlerine göre yapılır; gece yarısını aşan aralıklar (örn. 23:00 → 07:00) desteklenir.

Durum geçişleri: (1) NORMAL → PRE_ALARM: hareketsizlik süresi dinamik eşiği aştığında, kullanıcıya inactivity_pre_alarm eventi ile geri sayımlı bildirim gönderilir. (2) PRE_ALARM → CONFIRMED: 60 saniye (ayarlanabilir) içinde yanıt gelmezse alarmType: "inactivity" ile alarm oluşturulur ve emergency_alert eventi emit edilir. (3) PRE_ALARM → NORMAL: hareket tespit edilirse (varyans > 0.1 g²) veya kullanıcı inactivity_cancel eventi gönderirse otomatik iptal edilir.

Hareket tespiti: sensorAnalyzer.js modülü, Redis'te tuttuğu kayan pencere buffer'ına her sensör verisi geldiğinde ivme büyüklüğü ekler ve popülasyon varyansını hesaplar. Varyans > 0.1 g² ise hareket var kabul edilir.

6.5 Alarm ve Bildirim Modülü

Düşme onayı (FALL_CONFIRMED) veya hareketsizlik onayı (CONFIRMED) gerçekleştiğinde sistem otomatik olarak bir Alarm kaydı oluşturur ve Socket.IO üzerinden fall_detected / emergency_alert event'leri ile mobil uygulamaya ve web paneline bildirim gönderir. Mobil uygulamada düşme tespiti sonrası 10 saniyelik geri sayımlı bir alarm ekranı gösterilir; kullanıcı süre dolmadan „İyiym (İptal Et)“ butonuna basabilir.

Düşme alarmında, kullanıcı iptal ettiğinde mobil uygulama hem fall_cancel eventini emit eder hem de PATCH /api/alarms/:id/resolve isteğiyle alarmı veritabanında isResolved: true olarak işaretler. Backend, fall_cancel eventini aldığı anda web paneline alarm_resolved bildirimi gönderir; Redis'teki düşme durum bilgisi ise zaten kısa TTL'li olduğu için ayrıca temizlenmesine gerek kalmaz.

6.6 Socket.IO Gerçek Zamanlı İletişim Modülü

Socket.IO bağlantıları JWT token ile doğrulanır; token, bağlantı kurulurken handshake.auth.token alanından okunur. Kimlik doğrulama başarılı olduğunda kullanıcı bilgileri (profil tipi, uyku takvimi, rol) veritabanından çekilerek socket nesnesinde cache'lenir; böylece her sensör penceresinde tekrar veritabanı sorgusu yapılmaz.

Oda yönetimi: user:{userId} odasına her kullanıcı kendi bildirimlerini almak için katılır; panel:{userId} odasına ise admin kullanıcılar otomatik olarak katılır ve panel güncellemelerini alır.

Event	Yön	Açıklama
-------	-----	----------

sensor_window	Mobil → Backend	Sensör penceresi verisi
fall_detected	Backend → Mobil + Panel	Düşme tespit/onay bildirimi
fall_cancel	Mobil → Backend	Kullanıcı düşme alarmını iptal etti
alarm_resolved	Backend → Panel	Alarm çözüldü bildirimi
inactivity_pre_alarm	Backend → Mobil	Hareketsizlik ön alarmı (geri sayım)
inactivity_cancelled	Backend → Mobil	Hareketsizlik ön alarmı otomatik iptal
inactivity_cancel	Mobil → Backend	Kullanıcı hareketsizlik alarmını iptal etti
emergency_alert	Backend → Mobil + Panel	Acil durum alarmı (hareketsizlik onaylı)
device_status	Backend → Panel	Cihaz anlık durumu
join_panel_room	Panel → Backend	Web panel oda katılımı (admin otomatik katılır)

6.7 Web Yönetim Paneli (Admin Dashboard) Modülü

Admin dashboard, React 18 + Vite + Tailwind CSS ile geliştirilmiş bir tek sayfa uygulamasıdır (SPA). ProtectedRoute bileşeni ile admin rolü kontrolü yapılır. Panel beş ana sayfadan oluşur:

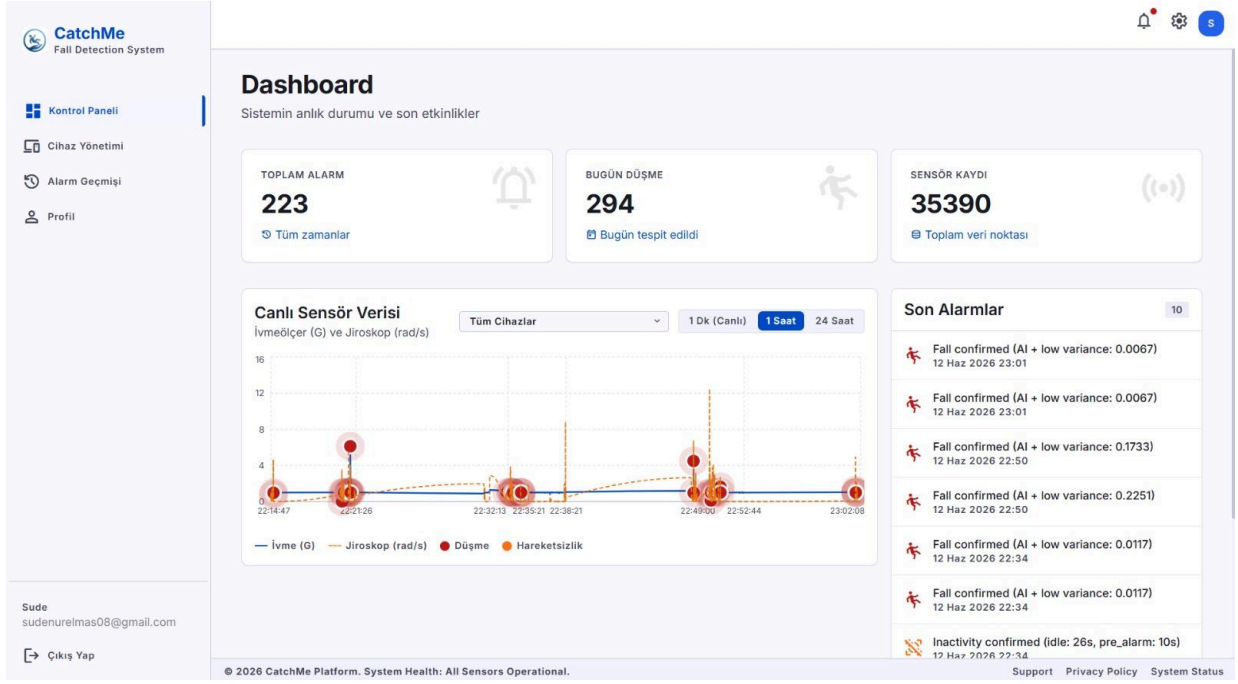
6.7.1 Giriş Sayfası (LoginPage)

Admin kullanıcıların JWT token ile kimlik doğrulaması yaptığı sayfadır. Token ve kullanıcı bilgileri AuthContext ile uygulamanın her yerinde erişilebilir kılınır. Admin olmayan kullanıcılar frontend tarafında da engellenir.

6.7.2 Dashboard Sayfası (DashboardPage)

Sistemin genel durumunu özetleyen ana sayfadır. Üst kısımda üç istatistik kartı gösterilir: Toplam Alarm, Bugünkü Düşmeler ve Sensör Kaydı. Alt kısımda SensorChart bileşeni ile

gerçek zamanlı sensör grafikleri görselleştirilir; RecentAlarms bileşeni son 10 alarmı listeler.



Şekil 4: Web Arayüzü Dashboard Sayfası

6.7.3 Sensör Grafikleri (SensorChart)

Recharts kütüphanesi ile uygulanan zaman serisi grafikleridir:

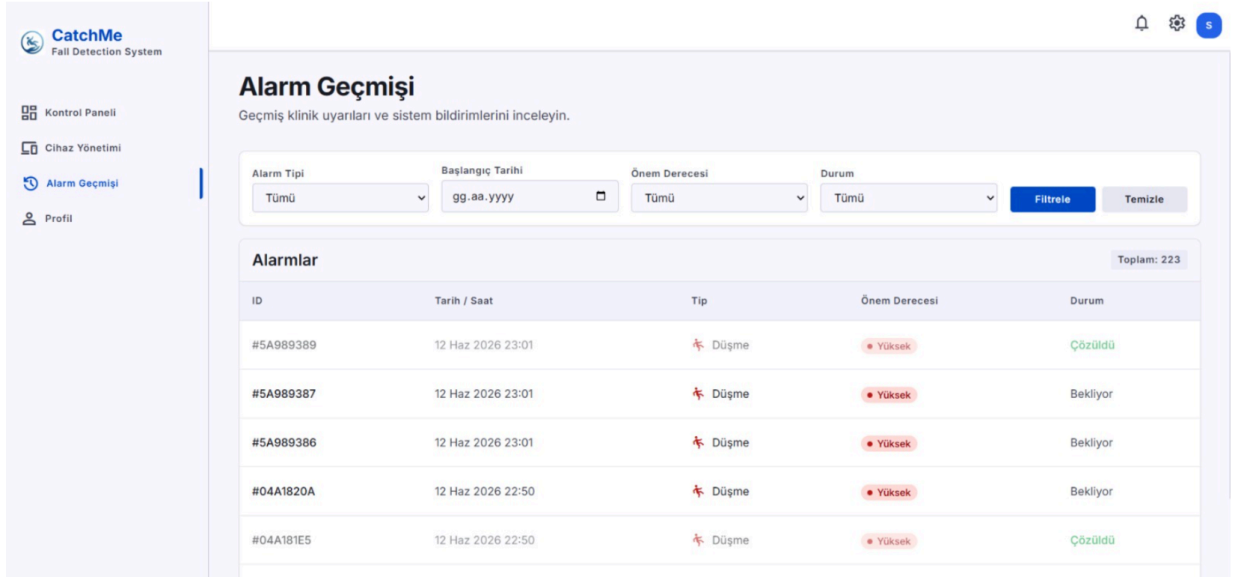
- İvmeölçer büyüklüğü (accelerometer.magnitude) ve jiroskop büyüklüğü (gyroscopeMagnitude, backend tarafından önceden hesaplanır) zaman içindeki değişimleriyle gösterilir.
- Görüntüleme modları: canlı (son 60 saniyelik kayan pencere), 1 saat ve 24 saat.
- Cihaza göre filtreleme arayüzde mevcuttur.
- Grafik üzerinde özel bir hover/tooltip bileşeni ile veri noktaları detaylı olarak gösterilir; Socket.IO üzerinden gelen device_status event'leri ile grafik anlık olarak güncellenir.

6.7.4 Cihaz Listesi Sayfası (DevicesPage)

Sistemdeki tüm cihazları listeleyen sayfadır. Tablo kolonları: Cihaz ID, Durum (çevrimiçi/çevrimdışı — son 5 dakika içinde veri geldiyse çevrimiçi), Magnitude (son ivme büyüklüğü değeri), Düşme Sayısı ve Son Görülme zamanı.

6.7.5 Alarm Geçmişi Sayfası (AlarmHistoryPage)

Tüm alarm kayıtlarının listelendiği sayfadır. Alarmlar tipine (düşme/hareketsizlik), ciddiyetine ve çözüm durumuna göre filtrelenebilir; her alarm, ilişkili sensör verisiyle birlikte detay mesajıyla gösterilir.



Şekil 5: Web Arayüzü Alarm Sayfası

6.7.6 Profil Sayfası (ProfilePage)

Admin kullanıcının kendi profil bilgilerini görüntülediği ve güncelleyebildiği sayfadır. Bildirim tercihleri şu an tarayıcının yerel depolama alanında (localStorage) tutulmaktadır; bu tercihlerin backend'e taşınması ileri geliştirme olarak planlanmıştır.

6.7.7 Panel REST API Uç Noktaları

Web paneline özel olarak /api/panel öneki altında dört uç nokta tanımlanmıştır (tümü protect + adminOnly middleware'i ile korunur):

Metot	Yol	Açıklama
GET	/api/panel/stats	Toplam alarm, çözülmemiş alarm, bugünkü düşme, toplam sensör kaydı, toplam kullanıcı
GET	/api/panel/recent-alarms	Son 10 alarm (çözülmemiş öncelikli, SensorData populate)
GET	/api/panel/sensor-chart	Sensör zaman serisi verisi (hours, deviceId, userId filtreli)
GET	/api/panel/devices	Cihaz listesi (aggregation: lastSeen, magnitude, fallCount, isOnline)

6.8 AI Mikroservisi Modülü

AI mikroservisi, Node.js backend'den bağımsız çalışan bir Python/FastAPI uygulamasıdır ve Uvicorn ASGI sunucusu üzerinde http://127.0.0.1:8000 adresinde hizmet verir.

Metot	Yol	Açıklama
GET	/health	Servis sağlık kontrolü (model yükleme durumu, eşik değeri)

POST	/predict	Düşme tahmin isteği (SensorWindowPayload → PredictResponse)
------	----------	---

Model yüklenemezse veya tahmin sırasında hata oluşursa servis güvenli varsayılan döner; health endpoint'i bu durumda "degraded" raporlar.

7. API Dokümantasyonu

Tüm API uç noktaları /api öneki ile erişilebilir. JWT gerektiren uç noktalarda Authorization: Bearer <token> başlığı zorunludur.

7.1 Kimlik Doğrulama (/api/auth)

Metot	Yol	Açıklama	Kimlik Doğrulama
POST	/api/auth/register	Yeni kullanıcı kaydı	✗
POST	/api/auth/login	Giriş yapma ve token alma	✗
GET	/api/auth/me	Oturum açmış kullanıcı bilgisi	✓
PATCH	/api/auth/me	Profil güncelleme (profileType, sleepSchedule, acil durum kişisi)	✓

7.2 Sensör Verisi (/api/sensor-data)

Metot	Yol	Açıklama	Kimlik Doğrulama
POST	/api/sensor-data	Yeni sensör verisi oluşturma (kural tabanlı düşme kontrolü dahil)	✓
GET	/api/sensor-data	Sensör verisi listeleme (sayfalı, filtreli)	✓
GET	/api/sensor-data/latest	En son sensör verisi	✓
GET	/api/sensor-data/falls	Düşme tespit edilen veriler	✓

Desteklenen sorgu parametreleri: page, limit, deviceId, isFallDetected, startDate, endDate.

7.3 Alarm Yönetimi (/api/alarms)

Metot	Yol	Açıklama	Kimlik Doğrulama
GET	/api/alarms	Alarm listesi (admin için tüm kullanıcıların alarmları, standart kullanıcı için kendi alarmları)	✓
PATCH	/api/alarms/:id/resolve	Alarmı çözüldü olarak işaretle	✓

7.4 Admin Uç Noktaları (/api/admin)

Metot	Yol	Açıklama	Yetkilendirme
GET	/api/admin/users	Tüm kullanıcıları listele	Admin
GET	/api/admin/falls	Tüm düşme kayıtları (sayfalı)	Admin
GET	/api/admin/dashboard	Dashboard özet istatistikleri	Admin
GET	/api/admin/example	Admin erişim testi	Admin

7.5 Web Panel Uç Noktaları (/api/panel)

Metot	Yol	Açıklama	Yetkilendirme
GET	/api/panel/stats	Panel istatistik kartları	Admin
GET	/api/panel/recent-alarms	Son 10 alarm (populate'li)	Admin
GET	/api/panel/sensor-chart	Sensör zaman serisi (1–24 saat)	Admin
GET	/api/panel/devices	Cihaz listesi (aggregation)	Admin

sensor-chart sorgu parametreleri: hours (1–24), deviceId, userId.

7.6 AI Servisi Uç Noktaları (Port 8000)

Metot	Yol	Açıklama
GET	/health	Servis sağlık kontrolü
POST	/predict	Düşme tahmini (SensorWindowPayload → PredictResponse)

7.7 Sağlık Kontrolü

Metot	Yol	Açıklama
GET	/health	Backend sağlık kontrolü
GET	/api/health	Backend API sağlık kontrolü

8. Test Süreci

Projenin kalite güvencesi, bileşen bazlı otomatik test süitleriyle sağlanmıştır. Backend testleri Jest, Supertest ve bellekte çalışan izole bir MongoDB sunucusu (mongodb-memory-server) ile geliştirilmiştir; her test süiti üretim veritabanından bağımsız, temiz bir ortamda çalıştırılır. Backend'e ek olarak web paneli Vitest ile, mobil uygulama jest-expo ile, AI servisi ise pytest ve httpx ile test edilmektedir.

8.1 Backend Test Sonuçları

Metrik	Beklenen	Gerçekleşen
Test Süiti Sayısı	8	8 / 8 geçti
Toplam Test Sayısı	65	65 / 65 geçti
Başarısız Test	0	0 (tüm testler geçti)

Kapsam Türü	Oran	Hedef	Durum
Statements (İfadeler)	91.52%	≥ %90	Geçti
Branches (Dallar)	83.67%	≥ %80	Geçti
Functions (Fonksiyonlar)	91.89%	≥ %90	Geçti
Lines (Satırlar)	91.52%	≥ %90	Geçti

8.2 Diğer Bileşenler İçin Test Kapsamı

Backend tarafında geliştirilen 65 otomatik test başarıyla geçmekte ve genel kod kapsama oranı %91'in üzerinde seyretmektedir. Backend testlerine ek olarak, web panelinde Vitest ile yaklaşık 40 senaryo, mobil uygulamada jest-expo ile yaklaşık 34 senaryo ve AI servisinde pytest ile yaklaşık 50 test koşulmaktadır.

9. Karşılaşılan Kısıtlar ve Geliştirme Önerileri

9.1 Mevcut Kısıtlar

- Push Bildirim / SMS Entegrasyonu: Acil durum kişilerine otomatik bildirim (Firebase Cloud Messaging, Twilio vb.) henüz entegre edilmemiştir. Sistem şu an yalnızca uygulama içi Socket.IO bildirimi sunmaktadır.
- AI Modeli Tipi: Kullanılan model Random Forest olup, deep learning tabanlı bir modele (LSTM, CNN gibi zaman serisi modelleri) kıyasla temporal pattern yakalama kapasitesi sınırlıdır.

9.2 Gelecek Geliştirme Önerileri

1. RiskAnalyzer.js modülünün hareketsizlik süresi, düşme sıklığı ve gece aktivitesi gibi faktörleri birleştiren çok boyutlu bir risk skoru üretecek şekilde geliştirilmesi.
2. Push notification veya SMS entegrasyonu (Firebase Cloud Messaging ya da Twilio) ile acil durum kişilerine otomatik bildirim gönderilmesi.
3. Derin öğrenme modeli (LSTM/CNN) ile zaman serisi tabanlı düşme tespitinin denenmesi.
4. Admin panelinin özelleştirilmesi, profil sayfasındaki bildirim tercihlerinin gerçekleştirilmesi

10. Sonuç

Akıllı telefon sensörlerini IoT uç düğümü olarak kullanan gerçek zamanlı bir düşme ve hareketsizlik tespiti platformu tasarlanmış ve hayata geçirilmiştir.

Tamamlanan İleri Düzey Özellikler

- AI Entegrasyonu: Python/FastAPI tabanlı bağımsız bir makine öğrenmesi mikroservisi geliştirilmiş; SisFall veriseti örnek alınarak eğitilmiş Random Forest modeli, ivmeölçer ve jiroskop verilerinden çıkarılan 6 istatistiksel öznitelik üzerinden düşme tahmini yapmaktadır. Model %82 olasılık eşiği ile çalışmakta, erişilemezlik durumunda kural tabanlı fallback otomatik devreye girmektedir.
- Jiroskop Entegrasyonu: Jiroskop verisi rad/s $\rightarrow$ °/s dönüşümüyle AI modeline girdi olarak beslenmekte ve web panelinde ayrı bir grafik serisi olarak görselleştirilmektedir.
- Çok Aşamalı Düşme Doğrulaması: Redis tabanlı durum makinesi (NORMAL $\rightarrow$ IMPACT_DETECTED $\rightarrow$ FALL_CONFIRMED) ile AI tespiti sonrası varyans analizi yapılarak yanlış pozitifler azaltılmıştır.
- Hareketsizlik Tespiti: Kullanıcının uyku takvimine göre gece/gündüz dinamik eşiklerle çalışan, PRE_ALARM geri sayımı ve kullanıcı onayı mekanizmasına sahip tam işlevsel bir hareketsizlik izleme sistemi geliştirilmiştir.
- Web Yönetim Paneli: React + Vite + Tailwind CSS ile gerçek zamanlı admin dashboard geliştirilmiş; Recharts ile sensör verisi görselleştirme, cihaz izleme, alarm filtreleme/listeleme ve istatistik kartları içeren bir yönetim arayüzü sunulmaktadır.
- Redis Durum Yönetimi: Düşme durum makinesi ve sensör buffer'ı için TTL tabanlı otomatik temizleme, hareketsizlik durum makinesi için ise kalıcı anahtarlarla atomik durum geçişleri sağlanmıştır.
- JWT Kalıcılığı: Token mobil uygulamada AsyncStorage ile kalıcı olarak saklanmakta; ortam değişkenleri .env dosyası ile yönetilmektedir.